

NETWORK INTRUSION DETECTION SYSTEM

Counselor NIDS

A counselors-based intrusion detection architecture with machine-learning detectors, Snort signature analysis, live packet capture, and an operator dashboard. Complete system documentation.

Version main @ 3d2077f · 27 commits

Generated 2026-10-07

Based on Quincozes et al., *A Counselors-Based Intrusion Detection Architecture*, IFIP 2019

— Contents

01 Introduction & concept

02 System architecture

03 The detection engine

04 Classifiers & datasets

05 Live capture & flow metering

06 Live detectors & false-alarm engineering

07 Snort signature analysis

08 The operator dashboard

09 Backend API

10 Setup & operation

11 Results

12 Security & production

13 Limitations & honest assessment

14 Project layout & reference

01 Introduction & concept

Counselor NIDS is a network intrusion detection system that judges each network connection with two independent methods and lets them cover each other's blind spots.

The project began as a faithful reproduction of *A Counselors-Based Intrusion Detection Architecture* (Quincozes et al., IFIP 2019) in Python with scikit-learn, and grew into a complete working system: live packet capture, Snort integration, a distributed service runtime over Redis, and a web dashboard for operators.

The counselors idea

Detection is performed by a team of specialist **detectors**. Each detector learned to recognise certain kinds of attack. Internally it clusters traffic and, per cluster, uses only the classifiers that are most accurate there. When a detector's selected classifiers *disagree* on a connection, it does not guess — it asks the other detectors ("counselors") for advice, weighted by their historical accuracy, and adopts the best acceptable answer. The advised sample is then stored as a new training example, so detectors learn over time.

Two engines, by design

Alongside the machine-learning detectors, the system runs **Snort 3** — a widely used signature IDS — on the same traffic. The ML judges how a connection *behaves* (its size, timing and rates); Snort matches known attack *patterns*. Each covers what the other misses, which is the system's central reliability principle.

Design philosophy. Throughout, the system favours honesty over flattery: it reports what it genuinely detects, is explicit about its blind spots, and was hardened under real adversarial testing rather than only on clean datasets.

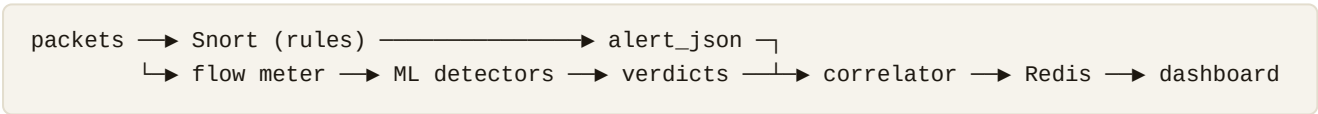
02 System architecture

The same detection logic runs in two modes: as a research library for offline experiments, and as a set of independent services communicating over Redis for live and replayed traffic — mirroring Figure 1 of the paper.

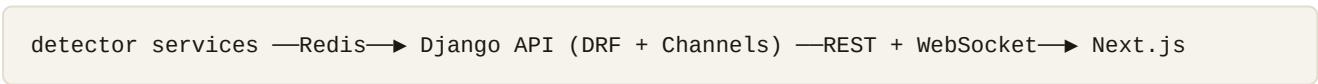
Distributed services (Redis)

Service	Role
<code>nids</code> <code>extract</code>	Extractor — replays a flow CSV, or captures live traffic, into the Unknown Samples stream.
<code>nids</code> <code>observe</code>	Observer — routes each batch to the detectors subscribed to its data source.
<code>nids</code> <code>detect</code>	Detector — classifies its inbox, requests advice from the others over Redis, answers their requests, optionally retrains.
<code>nids</code> <code>snort</code>	Snort bridge — runs Snort on the same traffic and correlates each alert to its flow.
<code>nids</code> <code>monitor</code>	Monitor — live counters: accuracy, detection rate, conflicts, advice, cross-checks.

Data flow



Attack decisions are published to the `nids:alerts` stream; Snort alerts (with the ML's verdict on the same flow) to `nids:snort:alerts`. The Django API reads these streams and relays them to the Next.js dashboard over one WebSocket.



Repository shape

Core logic lives in `src/nids/` (the library and the `nids` command); research scripts in `experiments/`; data pipelines in `scripts/`; the API in `backend/`; the dashboard in `frontend/`; Snort config in `snort/`. The full layout is in §14.

03 The detection engine

Algorithm 1 — offline model building

For each detector (`src/nids/detector/model.py`):

1. Train every candidate classifier on the training signatures.
2. Cluster the evaluation signatures with K-Means.
3. Measure each classifier's accuracy within each cluster.
4. Per cluster, select the most accurate classifier plus any within `alpha` of it.

Numeric features are log-compressed before scaling; without this, K-Means on CICIDS2017 isolates single outlier flows in their own clusters.

Algorithm 2 — online detection

Each unknown connection (`src/nids/detector/detector.py`) is assigned to its nearest cluster and classified only by that cluster's selected classifiers. If two or more were selected and they *disagree*, the connection is a **conflict** that needs advice.

The counselors network

`src/nids/counselor/network.py` resolves conflicts. A counselor answers from its own recent unambiguous decisions within a look-back time window, together with its historical accuracy. Advice at or above `min_accuracy` (default 0.9) is accepted, becomes the final verdict, and the sample is stored for retraining.

Extension: cross-check

The paper has a blind spot: a detector cannot raise a conflict about an attack class it never trained on, so it confidently calls those flows "normal". With `cross_check_normal=True`, every "normal" verdict is also checked with the counselors, and a confident "attack" answer overrides it. On CICIDS2017 this lifts accuracy from ~83% to **99.88%**. The documentation is candid that a plain "any detector says attack" baseline does as well here, because both detectors have near-zero false alarms — the real gain is shared knowledge between detectors.

Extension: fallback suppression

When a detector's classifiers disagree and no counselor can advise, the leftover "attack" guess (resolution `fallback`) is the main source of live false alarms. The `suppress_fallback` option treats such guesses as normal. It is off by default (so the paper's experiments are unchanged) and on for the live detectors. See §6 for the measured impact.

04 Classifiers & datasets

Classifiers

scikit-learn stands in for the paper's Weka classifiers (`src/nids/detector/classifiers.py`): k-NN, decision tree (J48/C4.5), REPTree, RandomTree, random forest, naive Bayes and AdaBoost. NBTree, ADTree and KStar have no scikit-learn equivalent and are omitted.

Datasets

Dataset	Scenario / use	Attack coverage
NSL-KDD	Scenario 1 (three heterogeneous detectors)	Classic KDD attack families
CICIDS2017	Scenario 2; detectors 1 & 2	DoS, DDoS, PortScan
CSE-CIC-IDS2018	Detector 3; source of the live-detector captures	Brute force, botnet, web attacks

Datasets live in `data/raw/` and are not committed. CICIDS2017 ML CSVs have no timestamps and NSL-KDD has none at all, so detectors in each scenario view the same records and advice is matched on `record_id` with a zero window; a 2-second window is supported for timestamped data.

Wider coverage — detector 3

Detectors 1 and 2 know DoS, DDoS and PortScan. Detector 3 adds brute force, botnet and web attacks from CSE-CIC-IDS2018 and joins the counselors network. Adding it leaves the CICIDS2017 test essentially unchanged (accuracy 99.59% → 99.58%, false alarms 0.17% → 0.17%). Infiltration is deliberately excluded: its flows resemble normal traffic in this dataset, and learning them raised false alarms from 0.17% to 6.2%.

05 Live capture & flow metering

Live traffic is turned into flows by the Python `cicflowmeter`, run through `src/nids/capture/flowmeter.py`, which works around two bugs in `cicflowmeter` 0.5.0: its CLI passes arguments in the wrong order, and a flow without forward packets crashes its flow-writing thread (which would silently stop live capture).

Critical caveat. The Python flow meter differs from the Java `CICFlowMeter` behind the published CSVs in 71 of 154 feature medians (packet lengths include headers; flags and windows are counted differently; flows split differently). **The CSV-trained detectors flag 0% of brute-force flows computed the live way.** Live mode and packet-capture replays therefore use separate **live detectors** trained on flows computed the same way — see §6.

Live capture requires raw-socket rights (`sudo` , or a `setcap` on the interpreter). Flows reach the detectors when their connection ends or times out, so expect a delay of up to a couple of minutes.

06 Live detectors & false-alarm engineering

Two live detectors are trained from the CSE-CIC-IDS2018 raw captures run through the Python flow meter (`scripts/live_detectors/`):

Detector	Knows
<code>live_dos</code> (Flood detector)	DoS: Hulk, GoldenEye, Slowloris, SlowHTTPTest
<code>live_access</code> (Break-in detector)	FTP brute force, SSH brute force, web attacks

Live adversarial testing exposed two false-alarm problems, both fixed at inference time (the trained models are unchanged, and the paper's experiments are untouched). Both fixes were verified on all 339,598 labelled live flows.

Fix 1 — empty-flow filter

A port scan creates tens of thousands of connection-only flows (SYN/RST, no payload). The ML never trained on such flows and flagged them as best-guess "attacks" — one scan produced roughly 50,000 false alarms.

`carries_payload()` in `src/nids/services/live_capture.py` keeps flows that exchanged no application data out of the ML, while still indexing them so Snort (which detects scans) can link its alerts.

Proven safe. Across all 339,598 labelled flows — every attack type *and* benign — **0** carry no payload, so the filter removes 0% of what the detectors are trained to catch while eliminating the scan-noise flood. Opt out with `--keep-empty-flows`.

Fix 2 — fallback suppression

The remaining live false alarms (e.g. on normal HTTPS) were all low-confidence `fallback` guesses.

Measurement showed fallback accounts for just 16 of ~190,000 real detections, so suppressing it costs almost nothing:

On the live test set	Detection rate	False-alarm rate
Before (fallback kept)	99.38%	0.02%
After (<code>--suppress-fallback</code>)	99.37%	0.00%

Enabled for the live detectors in `start.sh` and packet-capture replays; off by default elsewhere.

Measured live-detector accuracy

Tested on the later 30% of each attack, by time (held out from training):

Test flows	<code>live_dos</code>	<code>live_access</code>	Counselor network
DoS SlowHTTPTest / FTP brute force	100% / 100%	100% / 99.99%	100% / 100%
DoS Hulk / GoldenEye / Slowloris	99.8% / 98.9% / 94.9%	23% / 10% / 0%	99.7% / 98.9% / 93.6%
SSH brute force / web attacks	0% / 0%	99.6% / 100%	99.6% / 89.4%
Benign (false alarms)	0.14%	0.02%	0.02%

07 Snort signature analysis

Snort 3 analyses the same packets as the ML. Every alert is linked to the flow it belongs to, and the ML gives its verdict on that flow through the same advice request detectors use with each other

(`src/nids/services/snort_bridge.py` , `flow_index.py`).

Agreement	Meaning
Confirmed	Snort flagged it and the ML says attack.
Disputed	Snort flagged it, the ML says normal (possible false alarm, or an attack the ML never trained on).
ML only	The ML flagged a flow Snort had no alert for.

Linking is by connection (both IPs, ports, protocol, either direction) and time: the alert must fall inside the flow's start-to-end window. Host-level alerts (a port scan stands for many probe connections) are linked to every flow between the two hosts within ± 30 s.

Rule set & tuning

The project ships 34 custom rules (`snort/rules/nids.rules`): port scans/sweeps, SQL injection, XSS, path traversal, SYN/HTTP floods, and FTP/SSH brute force. Inspectors' built-in protocol-anomaly alerts are off (they flagged mid-stream TLS as attacks — 35% of normal flows on the real capture); only the port-scan alerts (gid 122, except "open port") are on, and scan reports from port 53 are ignored (DNS answers looked like a UDP port scan).

Snort 3 community rules

`./start.sh setup` also downloads the 4,017 Snort 3 community rules (exploits, malware, C2, policy) into `snort/rules/community/` , loaded when present; `NIDS_SNORT_COMMUNITY=0` opts out.

Low-and-slow brute force

The FTP/SSH brute-force rules use a wide 120-second window (10 and 15 connections) so paced guessing designed to evade a short-window threshold is still caught; a fast burst falls inside the same window, so this only adds coverage. Verified: it flags a 12-attempt-over-88 s brute force that a 20-in-10 s rule misses, fires on none of a benign FTP session, and on `real-attacks-2018.pcap` flags only the attackers, not the benign workstation.

Whole-system detection on real traffic

On `real-attacks-2018.pcap` — real CSE-CIC-IDS2018 traffic from the late part of each attack, which the live detectors never trained on:

Traffic	Flows	ML (live)	Snort	Either
DoS GoldenEye	2,046	100.0%	4.4%	100.0%
DoS Hulk	4,153	99.6%	30.5%	99.6%
DoS SlowHTTPTest	4,450	100.0%	99.6%	100.0%
DoS Slowloris	1,352	56.7%	0.0%	56.7%
FTP brute force	5,040	100.0%	99.6%	100.0%
SSH brute force	359	98.3%	76.3%	98.9%
Normal (false alarms)	1,768	0.0%	0.0%	0.0%

The ML confirms 11,077 of Snort's 11,079 alerts and catches the DoS attacks Snort's rules miss; Snort adds its payload rules. Slowloris is the weak spot and varies between runs, because the flow meter times flows out by wall clock.

08 The operator dashboard

A Next.js dashboard shows detection as it happens, written for a non-expert ("AI" and "rule checker" rather than ML and Snort; "connection" for flow). Live data arrives over one WebSocket shared by all pages.

Page	Contents
Overview	Plain-language verdict with a severity breakdown, headline numbers, the activity timeline paired with the ranked Top risks panel, attack-type and Snort-rule breakdowns, top sources, latest alerts.
Alerts	Every flagged connection with a severity and a plain explanation; an Attackers view grouping alerts by source into ranked "attack stories" with behavioural threat scoring; full-history search; Print/PDF report.
AI detectors	Per-detector counters, throughput, and how decisions were made (alone / counselor advice / cross-check / fallback).
Test results	The experiment comparisons, coverage and self-learning curves.

Analytical features

- **Severity scoring** — each alert gets a 0–100 score (attack type, corroboration by both engines, breadth) and a Critical/High/Medium/Low tier.
- **Attacker correlation** — alerts are grouped by source IP into "attack stories" (e.g. *scan* → *brute force*), with behavioural threat scoring that elevates sources which escalate, persist, or hit many targets.
- **Offline IP reputation** — known-bad source IPs are badged against local blocklists (`scripts/fetch_blocklists.py`); the lookup is local, so no address leaves the machine.
- **Notifications** — desktop + sound alerts on new Critical/High incidents.
- **Response** — a "Block this IP" action generates a ready-to-paste firewall command (never runs anything privileged; warns on local addresses).

The interface uses a warm-dark "data journal" aesthetic: an amber accent, Space Grotesk and JetBrains Mono, monospace micro-labels, and editorial KPI cards.

Deliberately excluded. External threat-intel APIs (which would send the network's IPs to a third party) and ML anomaly scoring (which would risk reviving the false-alarm problem that was just fixed). Both are documented as explicit opt-ins rather than silently added.

09 Backend API

A Django application (DRF + Channels) bridges the Redis services and the dashboard. It starts a replay by launching the Observer, one detector service per model in `models/`, and an Extractor as subprocesses — the same services as the command line.

Endpoint	Purpose
POST <code>/api/auth/login refresh logout/</code>	Cookie session (httpOnly, SameSite=Strict).
POST <code>/api/auth/token/</code> , <code>/token/refresh/</code>	JWT for scripts (Bearer header).
GET <code>/api/detectors/</code>	Live counters per detector + replay state.
GET <code>/api/alerts/?limit=&after=</code>	Latest attack decisions.
GET <code>/api/incidents/?source=&q=&limit=&offset=</code>	Flagged flows with ML and Snort verdicts, over the full history.
GET <code>/api/results/</code>	Experiment comparisons and self-learning curves.
GET <code>/api/reputation/?ips=</code>	Which IPs are on the local blocklists (offline).
GET <code>/api/replay/</code> , POST <code>.../start stop/</code>	Replay control.
ws://.../ws/live/	Stats every second, new alerts, reset on a new replay.

Settings come from environment variables (or `backend/.env`): `DJANGO_SECRET_KEY` , `DJANGO_DEBUG` , `DJANGO_ALLOWED_HOSTS` , `CORS_ALLOWED_ORIGINS` , `NIDS_REDIS_URL` , `NIDS_ROOT` , `DJANGO_DB_PATH` .

10 Setup & operation

One command

```
./start.sh setup          # once: env, packages, database, login user, models, demo
capture
./start.sh                # dashboard on http://localhost:3000
./start.sh live eth0      # Snort + ML on an interface, plus the dashboard (asks for
sudo)
./start.sh live capture.pcap # the same on a recorded capture
./start.sh --prod         # production servers (Daphne, optimised build, DEBUG off)
```

Needs Python 3.11+, Node, Redis, and (for the Snort side) Snort 3. Ctrl+C stops everything; logs are in `logs/`.

Manual setup

```
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"          # add ",live" for live capture
pytest                          # unit + backend tests
```

Experiments

```
python experiments/tune.py scenario2 --fraction 0.2      # tune on validation (never test)
python experiments/evaluate.py scenario2                # evaluate on test, mean over
seeds
python experiments/self_learning.py --fraction 0.2      # streaming self-learning
python experiments/wider_coverage.py --skip infiltration --benign2017 0.2 --save
```

Distributed runtime (native)

```
nids train scenario2 --fraction 0.05
nids observe &
nids detect models/detector1.joblib --sources cicids2017 --cross-check &
nids detect models/detector2.joblib --sources cicids2017 --cross-check &
nids extract data/replay/scenario2.csv --source cicids2017 --wait-for 2
nids monitor --once
```

Docker (headless)

```
docker-compose run --rm train
docker-compose --profile headless up redis observer detector1 detector2 extractor monitor
```

Snort is not in the Docker image (Debian has no package), so Docker pcap replays run with the ML only.

Live detectors & blocklists

```
python scripts/live_detectors/fetch_captures.py # ~1.3 GB of CSE-CIC-IDS2018 captures
python scripts/live_detectors/build_dataset.py # attack windows; Python flow meter; labels
python scripts/live_detectors/train.py # models/live/live_dos.joblib,
live_access.joblib
python scripts/fetch_blocklists.py # optional: offline IP reputation lists
```

11 Results

Scenario 2 — CICIDS2017 accuracy (test set, mean of 3 seeds)

Method	Detector 1	Detector 2
Paper, proposed solution (reported)	88.36%	74.57%
Ours, paper's method	83.01%	83.77%
Ours, + cross-check	99.88%	99.88%
Ours, "any detector says attack" baseline	99.89%	99.89%
Ours, best single classifier	82.98%	83.54%

Self-learning (20% sample, seed 0 — standalone accuracy)

Detector & attack	Frozen	+ cross-check	Conflicts only (paper)
Detector 2 on DoS Hulk (chunk 1)	5.66%	99.74%	73.33%
Detector 1 on PortScan (chunk 9)	32.51%	99.87%	32.18%

Wider coverage — detector 3 (CSE-CIC-IDS2018 test)

Traffic	Detectors 1+2	1+2+3
Bot	0.2%	99.9%
FTP / SSH brute force	0.0%	~100%
Web brute force / XSS / SQL injection	7–11%	44% / 77% / 31%
Benign (false alarms)	5.23%	5.24%

The distributed runtime gives 99.58% accuracy and 99.28% detection rate on the 42,112 replayed flows.

12 Security & production

- The dashboard session lives in **httpOnly, SameSite=Strict cookies** (JavaScript never sees a token); state-changing requests also need an `X-NIDS-Client` header.
- Login is rate-limited to 10 attempts per minute per client.
- With `DJANGO_DEBUG=0` the API refuses to start without a real `DJANGO_SECRET_KEY` (32+ chars) and sends security headers.
- Beyond localhost: put an HTTPS reverse proxy in front and set `AUTH_COOKIE_SECURE=1` , `SECURE_SSL_REDIRECT=1` , `SECURE_HSTS_SECONDS` , `DJANGO_ALLOWED_HOSTS` and `CORS_ALLOWED_ORIGINS` .
- The web app never runs privileged commands; IP blocking is command-generation only, guarded against local/private addresses.
- Offline IP reputation downloads only public lists and uploads nothing.

13 Limitations & honest assessment

The system reliably detects the classic network attacks — scans, floods (DoS), password guessing (brute force) and common website attacks — with the ML judging behaviour and Snort matching known patterns, each covering the other's gaps. It is not a complete malware/APT defence, and is explicit about that.

- **No single layer is complete.** Live testing showed the two engines trading roles — one run Snort caught a brute force the ML missed; the next, the ML caught it and Snort's rule did not trip. They are a safety net *together*, not individually.
- **Novel attacks.** The ML only recognises what it was trained on; a genuinely new attack type is most likely to pass it. This is an unsolved problem industry-wide, and is why Snort's signatures run alongside.
- **Encrypted payloads** are opaque to both layers; the system reads behaviour and patterns, not packet contents.
- **Live DDoS, botnet and infiltration** have detectors only for the recorded-data mode, not the live detectors.
- **Validation.** The strongest results are on datasets and short adversarial tests. The honest next step is a multi-day live false-alarm run on the target network (`docs/live-testing.md` , §6).
- **Fidelity to the paper.** scikit-learn stand-ins for Weka classifiers and different data filtering mean absolute numbers are not directly comparable with the paper; Scenario 1 does not reproduce the paper's improvement.

Overall. As a research/learning/portfolio system it is complete, honest about its limits, and demonstrably improved under adversarial testing. As a production defence it is closer than most student projects but not finished: it needs sustained live validation and broader training data before being trusted unsupervised.

14 Project layout & reference

nids/	
— start.sh	one command to set up and run everything
— pyproject.toml	the nids package and its dependencies
— docker-compose.yml	container stack
— docs/	paper/, datasets.md, live-testing.md
— src/nids/	core library and the `nids` command
— datasets/	NSL-KDD, CICIDS2017, CSE-CIC-IDS2018 loaders
— capture/	pcap reading/slicing, remote-zip fetch, flow meter
— detector/	classifier selection & detection (Alg. 1 and 2)
— counselor/	advice exchange between detectors
— services/	Redis services: extractor, observer, detector,
	live capture, Snort bridge, flow index, reputation, monitor
— scenarios.py	scenarios with signature/validation/test splits
— evaluation.py	metrics and the paper's baselines
— cli.py	nids train extract observe detect snort monitor reset
— experiments/	tune, evaluate, self_learning, wider_coverage
— scripts/	make_synthetic_capture, live_detectors/, fetch_blocklists
— backend/	Django API (config, api/, auth/)
— frontend/	Next.js dashboard (app, components, lib)
— snort/	Snort 3 config and rules (+ community/)
— notebooks/	dataset exploration and result plots
— tests/	pytest suite, mirroring src/nids

CLI reference

Command	Purpose
nids train	Build and save detector models (+ a replay CSV of unseen flows).
nids extract	Replay a CSV, or capture live (--live), into the stream.
nids observe	Route batches to subscribed detectors.
nids detect	Run one detector service (--cross-check , --suppress-fallback , --retrain-every).
nids snort	Run/follow Snort and correlate alerts to flows.
nids monitor	Print live detector counters.
nids reset	Clear the Redis streams and counters.